

一、前缀和与差分

1.1 前缀和的基本概念

什么是前缀和？

前缀和是一种预处理技术，它把数组中的每个位置都存储为从开始到当前位置的所有元素的和。

基本思想：

- 原始数组： $a[1], a[2], a[3], \dots, a[n]$
- 前缀和数组： $s[i] = a[1] + a[2] + \dots + a[i]$

为什么有用？

有了前缀和数组，我们可以在 $O(1)$ 时间内求出任意区间 $[l, r]$ 的和：

text

$$\text{区间和} = s[r] - s[l-1]$$

1.2 前缀和的简单例子

假设数组： $[2, 3, 1, 4, 2]$

前缀和计算：

text

```
s[0] = 0
s[1] = 2
s[2] = 2 + 3 = 5
s[3] = 5 + 1 = 6
s[4] = 6 + 4 = 10
s[5] = 10 + 2 = 12
```

计算过后数组为： $[2, 5, 6, 10, 12]$

求区间 $[2, 4]$ 的和：

text

```
常规方法：  $3 + 1 + 4 = 8$ 
前缀和方法：  $s[4] - s[2-1] = 10 - 2 = 8$ 
```

例题1：简单的前缀和

题目：

有5个数字：2, 3, 1, 4, 2

查询3次：

- 第1到第3个数的和

- 第2到第4个数的和
- 第3到第5个数的和

代码实现：

```
#include <iostream>
using namespace std;

int main() {
    int a[6] = {0, 2, 3, 1, 4, 2}; // 下标从1开始
    int s[6] = {0}; // 前缀和数组

    // 计算前缀和
    for (int i = 1; i <= 5; i++) {
        s[i] = s[i-1] + a[i];
    }

    // 输出所有前缀和
    cout << "前缀和数组: ";
    for (int i = 1; i <= 5; i++) {
        cout << s[i] << " ";
    }
    cout << endl;

    // 查询
    cout << "第1到第3个数的和: " << s[3] - s[0] << endl;
    cout << "第2到第4个数的和: " << s[4] - s[1] << endl;
    cout << "第3到第5个数的和: " << s[5] - s[2] << endl;

    return 0;
}
```

E.G: [P8218 【深进1.例1】求区间和 - 洛谷](#)

1.3 差分的基本概念

什么是差分？

差分是前缀和的逆运算。如果前缀和是"求和"，那么差分就是"求差"。

基本思想：

- 原始数组: $a[1], a[2], a[3], \dots, a[n]$
- 差分数组: $d[i] = a[i] - a[i-1]$ (其中 $a[0] = 0$)

为什么有用？

使用差分数组，我们可以快速对原数组的某个区间进行加减操作。

区间修改技巧：

要给区间 $[l, r]$ 的所有元素加上 c ：

text

```
d[l] += c
d[r+1] -= c
```

例题2：简单的差分

题目：

数组初始为：[0, 0, 0, 0, 0]

进行3次操作：

- 给第2到第4个数加3
- 给第1到第3个数加2
- 给第3到第5个数加1

最后输出整个数组。

代码实现：

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int d[7] = {0}; // 差分数组，多开一些空间

    // 第一次操作：第2到第4个数加3
    d[2] += 3;
    d[5] -= 3;

    // 第二次操作：第1到第3个数加2
    d[1] += 2;
    d[4] -= 2;

    // 第三次操作：第3到第5个数加1
    d[3] += 1;
    d[6] -= 1;

    // 通过差分数组还原原数组
    int a[6] = {0};
    cout << "最终数组： ";
    for (int i = 1; i <= n; i++) {
        a[i] = a[i-1] + d[i];
        cout << a[i] << " ";
    }
    cout << endl;

    return 0;
}
```

E.G: [P2367 语文成绩 - 洛谷](#)

二、离散化

离散化详解

什么是离散化？

离散化是一种将大范围稀疏数据映射到小范围连续数据的技巧。

简单理解：就像给班级同学按照身高排序并编号，虽然大家身高各不相同，但我们只需要关心相对高低关系。

离散化的应用场景

- 数据范围很大（比如 $0 - 10^9$ ），但实际数据个数很少（比如1000个）
- 需要以数据值作为数组下标，但值太大无法直接使用
- 只关心数据的相对大小，不关心具体数值

离散化的基本步骤

1. **收集数据：**把所有需要离散化的数值收集起来
2. **排序：**对收集到的数值进行排序
3. **去重：**去除重复的数值
4. **建立映射：**为每个数值分配一个新的"编号"

例题3：计算排名

题目描述

给定一个数列，对每个元素计算它的排名。排名定义为：比该元素小的不同数字个数 + 1。

解题思路

1. 对原数组去重并排序，得到所有不同的数字
2. 对于原数组中的每个数字，在排序后的数组中查找它的位置
3. 位置编号 + 1 就是该数字的排名

代码实现

```
#include<iostream>
#include<algorithm>
using namespace std;
int a[100005],d[100005];//原数组和离散化数组

int main(){
    ios::sync_with_stdio(0);
    int T,n;
    cin>>T;
    while(T--){
        cin>>n;
        for(int i=1;i<=n;i++){
            cin>>a[i];
            d[i]=a[i];//同步原数组数据
        }
    }
}
```

```

    sort(a+1,a+n+1); //排序
    int cnt=unique(a+1,a+n+1)-(a+1); //去重
    for(int i=1;i<=n;i++){
        d[i]=lower_bound(a+1,a+cnt+1,d[i])-a; //归位
    }
    for(int i=1;i<=n;i++) cout<<d[i]<<" ";
    cout<<endl;
}
return 0;
}

```

E.G: [B3694 数列离散化 - 洛谷](#)

三、模拟高精度

3.1 高精度的基本概念

什么是高精度？

当数字太大，超过 `long long` 的范围时，我们需要用字符串或数组来存储数字，然后模拟手工计算的过程。

为什么要学高精度？

- 处理大数运算
- 理解计算机如何存储和计算大数
- 锻炼编程能力

存储方式：

通常把数字倒着存储，这样便于处理进位。

text

数字：12345

存储：a[0]=5，a[1]=4，a[2]=3，a[3]=2，a[4]=1

例题4：高精度加法

题目：

计算 $123456789 + 987654321$

代码实现：

```

/*
高精度的加法思想
1. 把大数存到字符串；

```

2. 字符串的每个字符数字都通过ASCII转换存到数组，
注意的是要低位存在数组开头： $a[i] = s[len-i-1] - '0'$;

3. 获取最大的数长度： $\max(len1, len2)$;

4. 把a,b值加入到c数组: $c[i] = a[i] + b[i]$;

5. c数组加法进位的算式:

① $c[i+1] += c[i] / 10$;

② $c[i] \% = 10$;

6. 数字溢出，长度+1;

7. 反向输出结果;

```
*/
#include<iostream>
#include<string>
using namespace std;
string s1,s2;
int a[10000],b[10000],c[100001];
int main(){
// 1.输入值，长度
cin>>s1>>s2;
int len1 = s1.size();
int len2 = s2.size();
// 2.把字符转为整数存到数组
// 注意要个位存到数组开头
for(int i=0;i<len1;i++){
    a[i] = s1[len1-i-1] - '0';
}
for(int i=0;i<len2;i++){
    b[i] = s2[len2-i-1] - '0';
}
// 3.获取最大的数。
int len = max(len1,len2);
// 对各个位数进行相加
for(int i=0;i<len;i++){
    c[i]=a[i]+b[i];
}
//4.进位
for(int i=0;i<len;i++){
    c[i+1] += c[i]/10;
    c[i] %= 10;
}
//5.溢出
while(c[len]==0 && len>0){
    len--;
}
if(c[len]>0){
    len++;
}
//6.反向输出
for(int i=len-1;i>=0;i--){
    cout<<c[i];
}
return 0;
}
```

E.G: [P1601 A+B Problem \(高精\)](#) - 洛谷

例题5：高精度乘法

代码实现：

```
#include <iostream>
#include <string>
using namespace std;
const int MAXN = 40500; // 最大长度
int a[MAXN], b[MAXN], c[MAXN];
int main() {
    string s1, s2;
    cin >> s1 >> s2;
    int n = s1.length(), m = s2.length(), len = n + m;
    // 逆序存储
    for (int i = 0; i < n; i++) a[n - i] = s1[i] - '0';
    for (int i = 0; i < m; i++) b[m - i] = s2[i] - '0';
    // 累加乘积
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            c[i + j - 1] += a[i] * b[j];
        }
    }
    // 处理进位
    for (int i = 1; i < len; i++) {
        if (c[i] >= 10) {
            c[i + 1] += c[i] / 10;
            c[i] %= 10;
        }
    }
    // 删除前导零并输出
    while (len > 1 && c[len] == 0) len--;
    for (int i = len; i > 0; i--) cout << c[i];
    return 0;
}
```

四、三分法

4.1 三分法的基本概念

什么是三分法？

三分法用于在单峰函数（先上升后下降，或先下降后上升）中寻找极值点。

为什么叫三分法？

因为每次把区间分成三份，通过比较两个中间点来缩小区间。

基本步骤：

1. 在区间 $[l, r]$ 内取两个点： $mid1 = l + (r-l)/3$ 和 $mid2 = r - (r-l)/3$
2. 比较 $f(mid1)$ 和 $f(mid2)$ 的大小

3. 根据比较结果缩小区间

例题6：寻找二次函数的顶点

题目：

给定 n 个二次函数 $f_1(x), f_2(x), \dots, f_n(x)$ (均形如 $ax^2 + bx + c$)，设 $F(x) = \max\{f_1(x), f_2(x), \dots, f_n(x)\}$ ，求 $F(x)$ 在区间 $[0, 1000]$ 上的最小值。

代码实现：

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const int MAXN=1e4+10;
const double eps=1e-9;
int a[MAXN],b[MAXN],c[MAXN],n;
double js(double x){
    double ans=INT_MIN; //负无穷，自己定义 -1000000000000
    for(int i=1;i<=n;i++) ans=max(ans,a[i]*x*x+b[i]*x+c[i]);
    return ans;
}
int main(){
    int t;
    scanf("%d",&t);
    while(t--){
        scanf("%d",&n);
        for(int i=1;i<=n;i++){
            scanf("%d %d %d",&a[i],&b[i],&c[i]);
        }
        double l=0,r=1000;
        while(r-l>eps){
            double lmid=l+(r-l)/3.0,rmid=r-(r-l)/3.0; //三分
            if (js(lmid)<js(rmid)) r=rmid;
            else l=lmid;
        }
        printf("%.4f\n",js(l));
    }
    return 0;
}
```

E.G: [P1883 【模板】三分 | 函数 - 洛谷](#)

总结

今天学习了四种重要的算法技巧：

1. 前缀和与差分

- 前缀和：快速求区间和
- 差分：快速进行区间修改

2. 离散化

- 把大范围稀疏数据映射到小范围连续数据

3. 模拟高精度

- 用数组存储大数
- 模拟手工计算过程

4. 三分法

- 在单峰函数中寻找极值点
- 每次把区间分成三份

这些技巧都是算法竞赛和编程中非常实用的工具，希望大家通过练习熟练掌握！